

Multi-Agent Systems in LangChain

Introduction to Multi-Agent Systems

1. Overview

Multi-agent systems enable large, complex tasks to be divided into smaller, specialised tasks managed by distinct agents. Instead of one general agent handling everything, LangChain allows the development of multiple expert agents, each specialising in a particular skill. This method reflects real-world teamwork—each agent concentrates on what it does best.

2. Why Multi-Agent?

A single LLM agent becomes overloaded when dealing with many tools or tasks. Multi-agent systems solve this by splitting responsibilities: - One agent may focus on data processing - Another on reasoning - Another on summarisation

This increases **accuracy**, **modularity**, and **control**.

3. Patterns in LangChain

LangChain supports two main multi-agent patterns: - **Supervisor Model (Tool-Calling Multi-Agent)**: A central agent controls the workflow and calls sub-agents like tools. - **Agent-to-Agent Handoff Model**: Agents can directly hand over control to other agents.

Both patterns are supported in LangChain via LangGraph for stable orchestration.

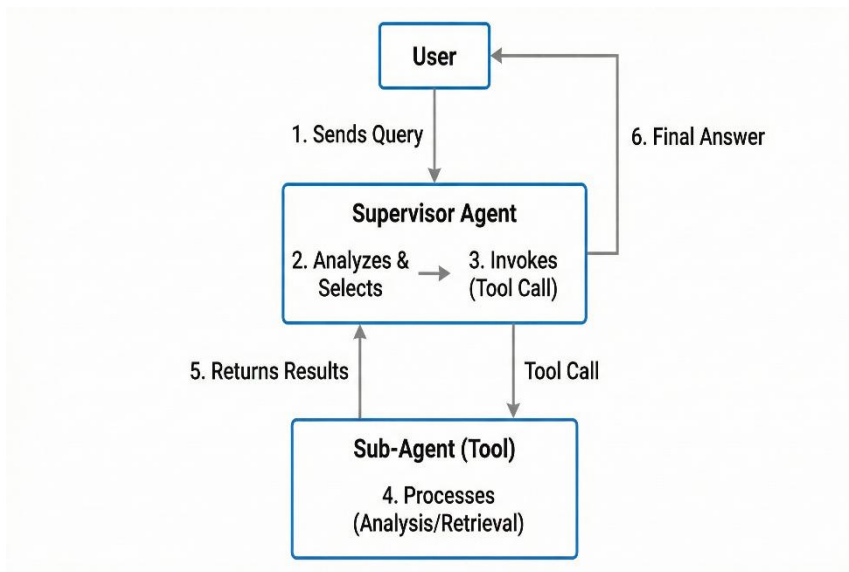
Supervisor Multi-Agent & Agent-to-Agent Handoff

4. Supervisor (Tool-Calling) Architecture

In this method, a central controller agent decides when to call each sub-agent.

Advantages: - Predictable workflow - Clear control - Easier debugging

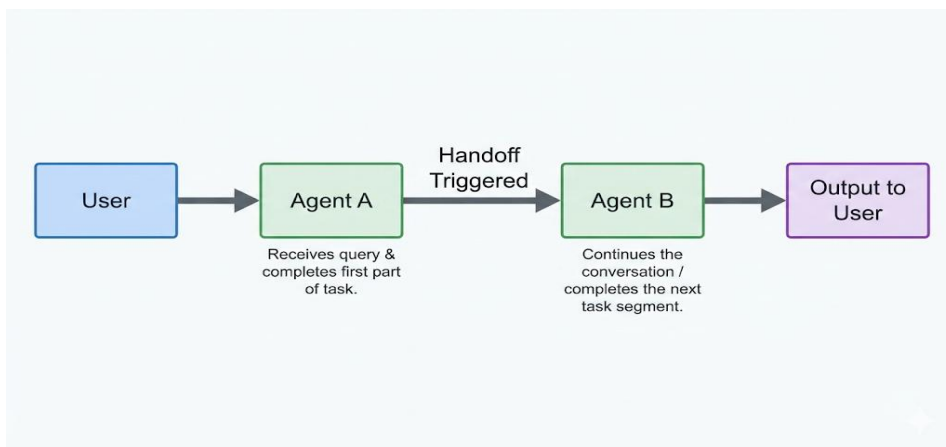
Flow:



5. Agent-to-Agent Communication (Handoff Model)

In this pattern, there is no central agent. Instead, one agent directly hands off control to another.

Flow:



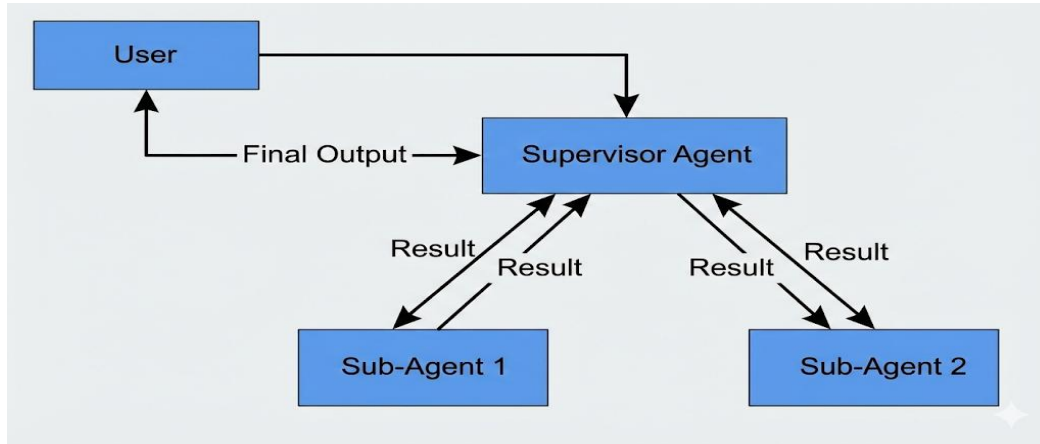
Advantages: - Feels natural in conversations - Supports multi-domain answers

When to Use: - When the user may need different experts for different queries - When conversation must flow between multiple domains (coding → ML → writing)

Architecture Summary & Key Concepts

6. Architecture Diagram (Conceptual Explanation)

Supervisor Model:



7. Key Design Concepts

1. **Roles** – Each agent specialises in one task.
2. **Tools** – Each agent has its own set of tools.
3. **Context Control** – You can choose what part of the conversation each agent sees.
4. **Memory Isolation** – Each agent maintains only relevant state.
5. **Routing Logic** – Determines which agent responds.

7. When to Choose Which Method

Pattern	Best For	Example
Supervisor Model	Structured workflows	Data analysis → Writing → Validation
Agent-to-Agent Handoff	Multi-domain conversation	Python help → ML help → Deployment help

8. Summary

Multi-agent systems in LangChain help break down large tasks into smaller units, each handled by a specialised agent. The two main approaches—Supervisor Model and Agent-to-Agent Handoff—provide flexibility for both structured workflows and conversational transitions. These models enable developers to construct scalable, modular AI systems that replicate real-world teamwork.